Linked List

- Learning Objectives
- In this session, you will be able to:

(a) describe Linked List ADT and demonstrate how it can be implemented from appropriate built-in types

(b) write algorithms to find an item in linked list

(c) write algorithms to insert an item in linked list

(d) write algorithms to delete an item from linked list

Linked List

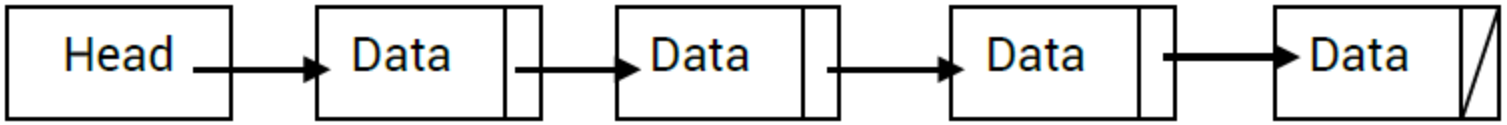
- It is a dynamic abstract data type that uses pointers to vary memory used at run time.
- A common question asked is why do we use linked lists when we can just use arrays?
- There are a number of benefits and drawbacks of using a linked list over an array.
- 1. The memory used by a linked list can vary at run time, meaning that memory isn't a fixed size (unlike the array)
- 2. In an array, memory spaces are continuous (even on the hard drive). So when you want to initialize an array, there has to be enough continuous free spaces on your hard-drive, and if not, then you will have to reduce the size of your array. Linked lists help tackle this problem, one part of the list can be on one end of the hard-drive and another part of the list on the other end of the hard-drive, the linked list will simply link both lists and display it as if the data were continuous.
- 3. Linked lists, can keep a track of non-continuous data, but this will slow down the access to the data. Arrays have very fast access to data.

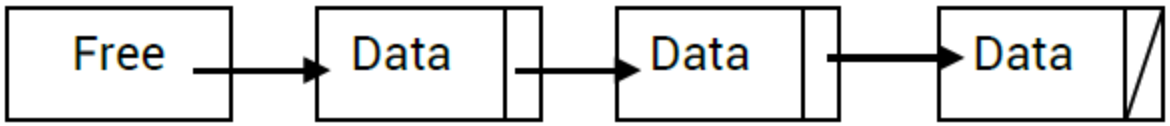
Linked List

- An element of a list is called a node.
- A node can consist of several data items and a pointer, which is a variable that stores the address of the node it points to.
- A pointer that does not point to anything is called a null pointer.
- A variable that stores the address of the first element is called a start (head) pointer.

Linked List

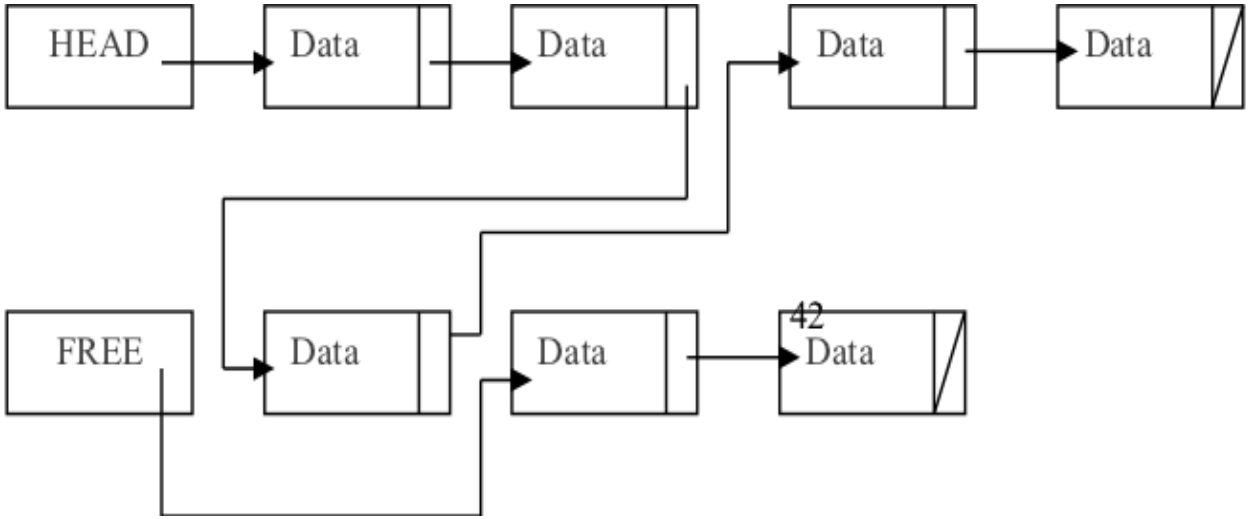
- Consider below Figure which shows a linked list and a free list. The linked list is created by removing cells/node from the front of the free list and inserting them in the correct position in the linked





Insertion

- Now suppose we wish to insert an element between the second and third cells in the linked list. The pointers have to be changed:



- The algorithm must check for an empty free list as there is then no way of adding new data. It must also check to see if the new data is to be inserted at the front of the list. If neither of these is needed, the algorithm must search the list to find the position for the new data.

Deletion

- Suppose we wish to delete the third cell in the linked list.

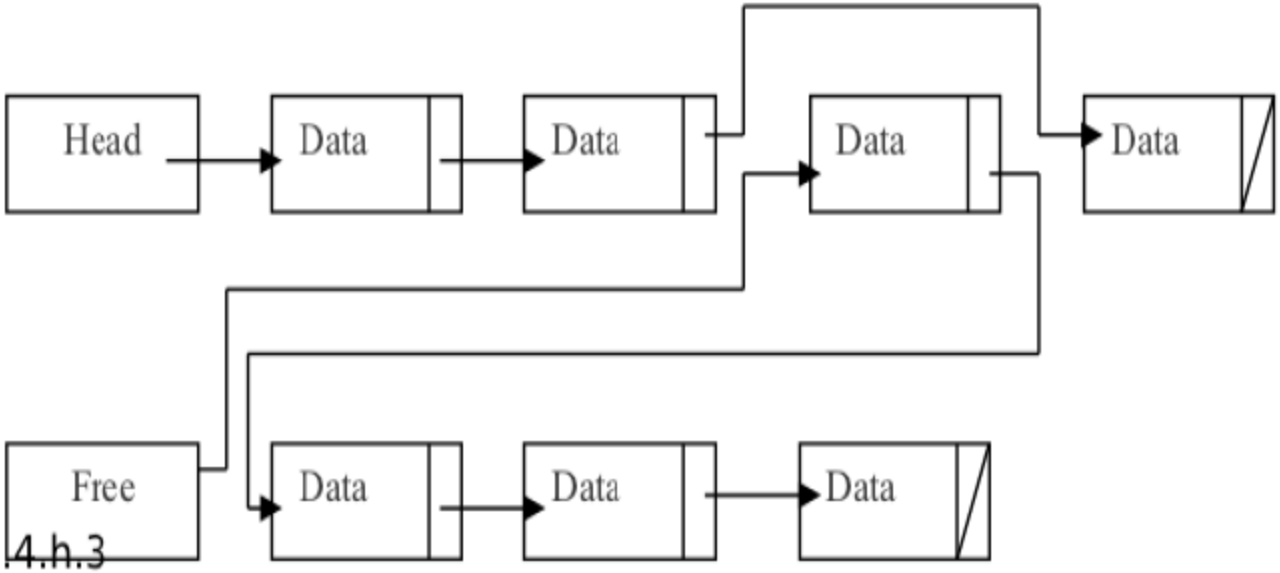


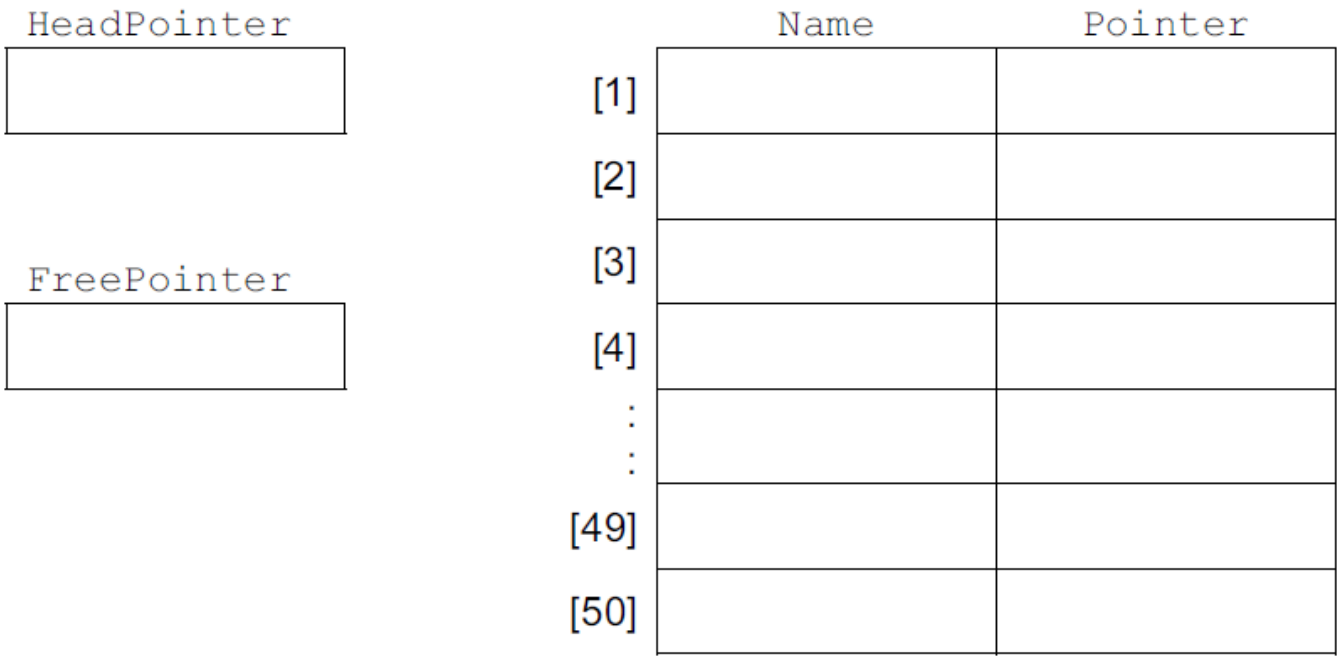
Fig. 3.4.h.3

Linear List Vs Linked List

- In real applications, the data would consist of much more than a keyfield and one data item. When list elements need reordering, only pointers need changing in a linked list.
- Unlike in a linear list, all data items would need to be moved.
- Using linked list saves time, however, we need more storage space for the pointer fields.

Linked List Implementation Using Array

- A linked list can be represented as TWO one-dimensional arrays:
  - A string array storing the data values, e.g. surname[20]
  - An integer array storing the indexes that create the links, e.g. link[20]
- The head pointer stores the index position of the first item in the list.



Create a new Linked List

- CONSTANT NullPointer = -1
- TYPE ListNode
  - DECLARE Data: STRING
  - DECLARE Pointer: INTEGER

  ENDTYPE

  DECLARE StartPointer: INTEGER

  DECLARE FreeListPtr: INTEGER

  DECLARE List[1:N] OF ListNode

```
PROCEDURE InitialiseList
    StartPointer =  NullPointer
    FreeListPtr = 0
    FOR Index = 0 TO N-1
        List[Index].Pointer = Index + 1
    ENDFOR
    List[N].Pointer = NullPointer
ENDPROCEDURE
```

Insert a new node

- PROCEDURE InsertNode(NewItem)
  - IF FreeListPtr <> NullPointer
    - THEN //There is space in the array
      - //Take node from free list and store data item
      - NewNodePtr = FreeListPtr
      - List[NewNodePtr].Data = NewItem
      - FreeListPtr = List[FreeListPtr].Pointer
      - //find insertion point
      - PreviousNodePtr = NullPointer
      - ThisNodePtr = StartPointer //start at beginning of list
      - WHILE ThisNodePtr <> NullPointer //while not end of list
        - AND List[ThisNodePtr].Data < NewItem
        - PreviousNodePtr = ThisNodePtr
        - ThisNodePtr = List[ThisNodePtr].Pointer

      ENDWHILE

Insert a new node (cont'd)

```
        IF PreviousNodePtr = NullPointer
          THEN     //insert new node at start of list
            List[NewNodePtr].Pointer = StartPointer
            StartPointer = NewNodePtr
          ELSE //insert new node after first node
            List[NewNodePtr].Pointer = List[PreviousNodePtr].Pointer
            List[PreviousNodePtr].Pointer = NewNodePtr
        ENDIF
    ELSE
        PRINT "No more space"
    ENDIF
ENDPROCEDURE
```

Find an element

- FUNCTION FindNode(DataItem) RETURNS INTEGER
  - CurrentNodePtr = StartPointer
  - WHILE CurrentNodePtr <> NullPointer //not end of list
    AND List[CurrentNodePtr].Data <> DataItem //item not found
    //follow the pointer to the next node
    CurrentNodePtr = List[CurrentNodePtr].Pointer
  - ENDWHILE
  - RETURN CurrentNodePtr
  - ENDFUNCTION

Deletion

- PROCEDURE DeleteNode(DataItem)

```
ThisNodePtr = StartPointer
WHILE ThisNodePtr <> NullPointer
                AND List[ThisNodePtr].Data <> DataItem
   PreviousNodePtr = ThisNodePtr
   ThisNodePtr = List[ThisNodePtr].Pointer
ENDWHILE
```

Deletion (cont'd)

```
IF ThisNodePtr <> NullPointer     //node exists in list
        THEN
            IF ThisNodePtr = StartPointer     //first node to be deleted
                    THEN
                        StartPointer = List[StartPointer].Pointer
                    ELSE
                        List[PreviousNodePtr].Pointer = List[ThisNodePtr].Pointer
            ENDIF
            List[ThisNodePtr].Poiner = FreeListPtr
            FreeListPtr = ThisNodePtr
    ENDIF
ENDPROCEDURE
```

Access All nodes in linked list

```
PROCEDURE OutputAllNodes
    CurrentNodePtr = StartPointer
    WHILE CurrentNodePtr <> NullPointer
        OUTPUT List[CurrentNodePtr].Data
        CurrentNodePtr = List[CurrentNodePtr].Pointer
    ENDWHILE
ENDPROCEDURE
```

- Note that a stack ADT and a queue ADT can be treated as special cases of linked lists.
- The linked list stack only needs to add and remove nodes from the front of the linked list.
- The linked list queue only needs to add nodes to the end of the linked list and remove nodes from the front of the linked list.